

Austin Software Development Process (SDP)

Principles

Great SDP so far! Here's what I would like to add to your paper. In the section of principles, you have covered most of the things that should be covered. You could elaborate on the code process from the beginning to the end. These include creating a branch for each new change as you have mentioned, then this branch would have to follow certain principles such as there should only be one person working on a branch at all times, and if there are dependencies they could branch off from your branch instead. The branch should also follow a common naming convention. During the development of the branch, we should keep committing every small goal that we've achieved instead of just coming at the very end. These commit messages should also be meaningful just like the branches. Once the development of the branch is finished, you could create a pull request ensuring that the creator have meaningful comments and explains the what where when why how of the new request. This request should then be able to pass all unit tests. This request should then be reviewed by at least another member of the team before merging it, that is you are not allowed to merge your own branch. Other than git related principles, we could talk about principles with our project partner Rahul, such as setting up timely meeting, being prepared in the meeting, taking notes during the meeting, and so on to ensure good communication. In regards to tasks, if you have any process, you should define their principles here too. There are things about tasks that you could talk about such as ensuring time based, meaningful, and doable tasks are made, and have it planned for a couple of weeks ahead.

Process

I like the iterative plan, but on the long run you should consider a proper process that is used in the development world to ensure that we are accomplishing our task within the time given. An iterative based approach is great in terms of flexibility, but then a lot of productivity is then sacrificed. Consider sprints, kanban, extreme programming, waterfall, agile, and others.

Roles

Great statement about roles! You could elaborate on the tasks for each role, you have written out that there are roles such as team leader and also a meeting scheduler like a secretary of some sort. As time went on, there will likely be the data engineer lead, the deployment lead, someone in charge of UI/UX, and depending on your chosen process, there are leadership roles to be taken there too, such as scrum master for scrums. For each of these, you could elaborate on their roles.

Tooling

This section is very clear, good job! You've hit all points except for the fact that there are things that we do not know as of yet, such as the VCS used, the project management software, the documentation tools that we will be using, test framework that we will be using, the CI/CD being used, but the rest seems to be whatever you'd desire and not up to Rahul's interpretation.

DoD

Great job here! The only thing I'd add is that we should all do a self review of our own code before asking for others to review it.

Release cycle

The release cycle here is pretty much what was given as skeleton. I would change certain things such as deploying to production on every release does not seem very customer friendly as there might be unstable code that could exist in staging but not allowed in production.

Environments

Great notes on the environments we should have, other than just having a good night's sleep, think about what it does for customers.